LAB 1: Python Environment Setup and Data Analysis

Objective

To install Python libraries using Anaconda and perform numerical and data analysis using NumPy and Pandas.

Procedure

Step 1:

Environment Setup:
The Anaconda Prompt executes the command ```conda install h5py matplotlib notebook pandas scipy seaborn```.

Installation Details:
The terminal lists packages such as bleach, cycler, h5py, matplotlib, and ipykernel that will be installed into the pyenv1 environment.

Process Insight:
The system resolves dependencies and prepares all required packages before installation.

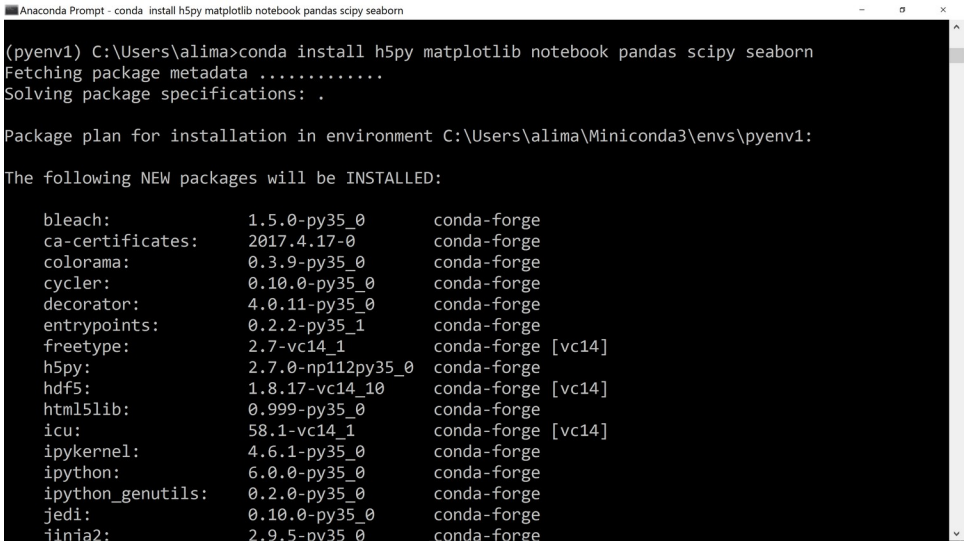

Figure 1.1

Step 2:

Installation Options:
The setup window shows installation options including adding Anaconda to PATH and registering it as default Python 3.9.

System Behavior:
A warning explains that adding Anaconda to PATH may cause conflicts.

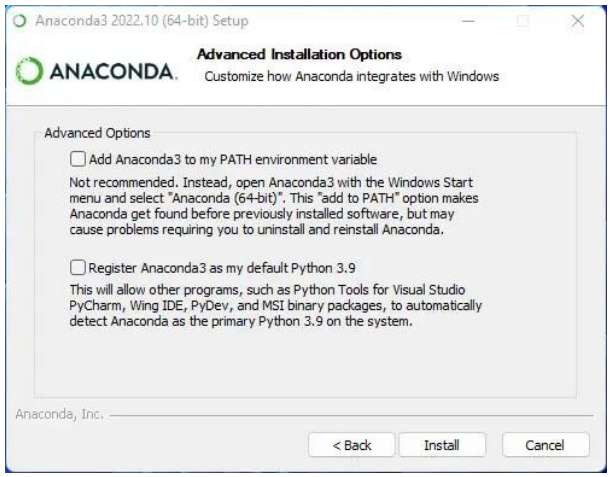

Figure 1.2

Step 3:

Interface Overview:
Anaconda Navigator displays tools including Jupyter Notebook, JupyterLab, Spyder, and VS Code.

Tool Access:
Each tool has a Launch button for direct access.

Purpose:
This dashboard allows managing environments and launching development tools.

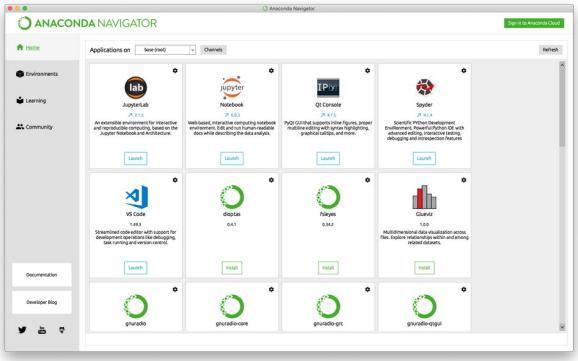

Figure 1.3

Step 4:

Version Check:
The terminal runs 'python --version' showing Python 3.13.5 and 'conda --version' showing conda 25.5.1.

Validation:
This confirms both Python and Conda are installed correctly.

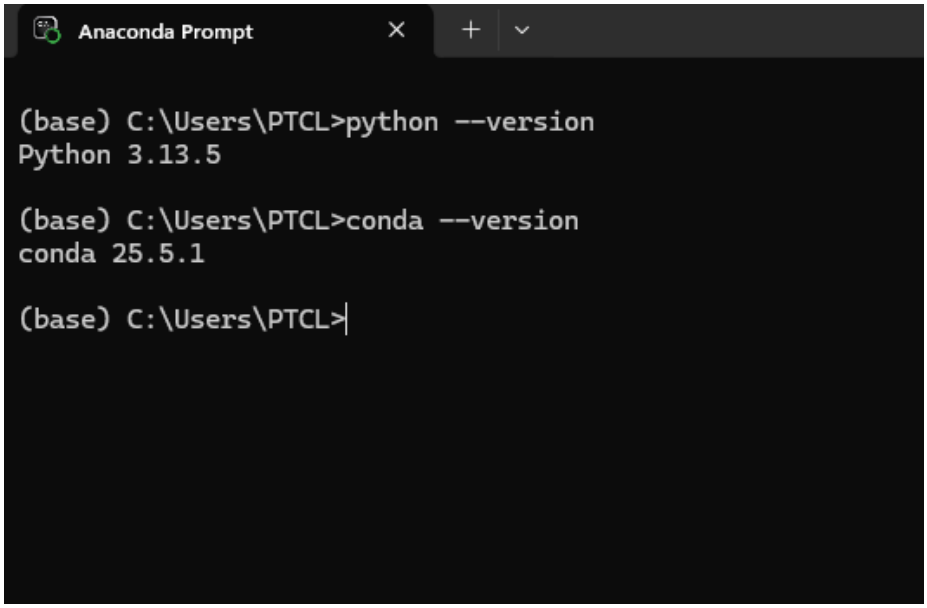

Figure 1.4

Step 5:

Command Execution:
The command 'pip install pandas numpy matplotlib seaborn' is executed.

System Response:
The output shows 'Requirement already satisfied' for all packages.

Interpretation:
All libraries are already installed and ready to use.

```
pip install pandas numpy matplotlib seaborn
```

```
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: seaborn in /usr/local/lib/python3.12/dist-packages (0.13.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.3)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.61.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.4.9)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
```

Figure 1.5

Step 6:

Library Installation:
The notebook installs ipdb using 'pip install -Uq ipdb'.

Code Implementation:
A Fibonacci function fib(n) is defined using iteration.

Execution Result:
Calling fib(8) returns 21.

Debug Status:
Automatic pdb calling is turned OFF.

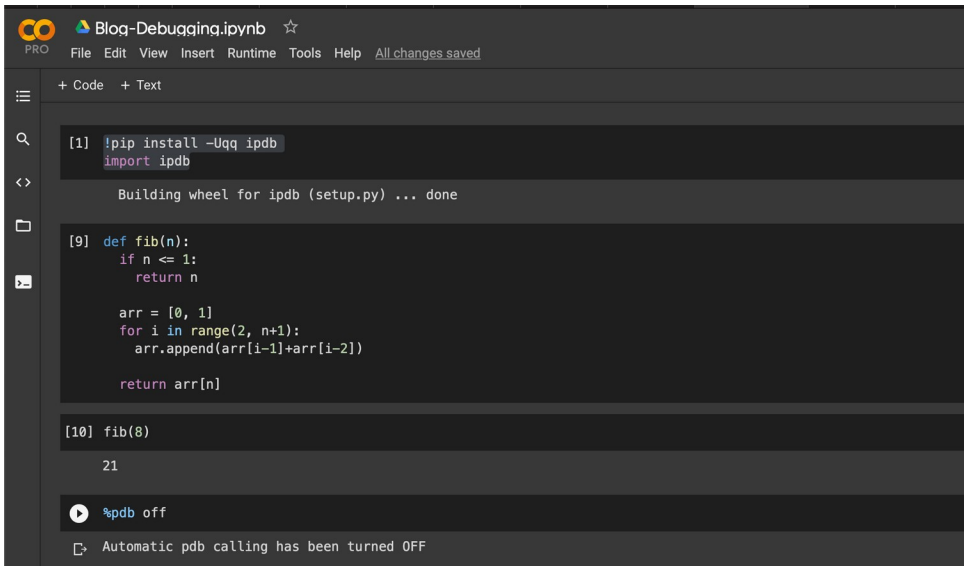

Figure 1.6

Step 7:

Setup:
It imports NumPy and creates an array [10, 20, 30, 40].

Batch Math:
The values are doubled to [20 40 60 80] and square roots are calculated.

Quick Stats:
Mean is 25.0 and maximum value is 40.

Insight:
NumPy processes all values simultaneously.

```python
import numpy as np

# Creating an array
arr = np.array([10, 20, 30, 40])


print(arr * 2)
print(np.sqrt(arr))

# Statistical functions
print(f"Mean: {np.mean(arr)}")
print(f"Max: {np.max(arr)}")
```

```
[20 40 60 80]
[3.16227766 4.47213595 5.47722558 6.32455532]
Mean: 25.0
Max: 40
```

Figure 1.7

Step 8:

Computation Output:
The NumPy operations are displayed again including array doubling and square root results.

Statistics:
Mean remains 25.0 and maximum value is 40.

Verification:
This confirms correct numerical computation.

Numerical Operations with NumPy

```python
import numpy as np

# Creating an array
arr = np.array([10, 20, 30, 40])


print(arr * 2)
print(np.sqrt(arr))

# Statistical functions
print(f"Mean: {np.mean(arr)}")
print(f"Max: {np.max(arr)}")
```

```
[20 40 60 80]
[3.16227766 4.47213595 5.47722558 6.32455532]
Mean: 25.0
Max: 40
```

Figure 1.8

Step 9:

Data Setup:
A dataset is created with students Zaid, Riyan, Tahar, Saima and scores 92, 85, 88, 95.

Data Structuring:
A DataFrame is created and a Grade column is added.

Output:
The table displays student names, scores, project status, and grades.

```python
import pandas as pd


data = {
    'Student': ['Zaid', 'Riyan', 'Tahar', 'Saima'],
    'Score': [92, 85, 88, 95],
    'Project_Completed': [True, True, False, True]
}
df = pd.DataFrame(data)

top_students = df[df['Score'] > 90]


df['Grade'] = ['A', 'B', 'B+', 'A+']

print(df)
```

	Student	Score	Project_Completed	Grade
0	Zaid	92	True	A
1	Riyan	85	True	B
2	Tahar	88	False	B+
3	Saima	95	True	A+

Figure 1.9

Step 10:

Filtering Logic:
Students with score greater than 90 are filtered using df[df['Score'] > 90].

Result:
Only high-scoring students are selected.

Display:
The table shows Student, Score, Project_Completed, and Grade.

Data Manipulation with Pandas

```python
import pandas as pd


data = {
    'Student': ['Zaid', 'Riyan', 'Tahar', 'Saima'],
    'Score': [92, 85, 88, 95],
    'Project_Completed': [True, True, False, True]
}
df = pd.DataFrame(data)

top_students = df[df['Score'] > 90]


df['Grade'] = ['A', 'B', 'B+', 'A+']

print(df)
```

```
  Student  Score  Project_Completed Grade
0    Zaid     92               True     A
1   Riyan     85               True     B
2   Tahar     88              False    B+
3   Saima     95               True    A+
```

Figure 1.10

Step 11:

Visualization Setup:
Matplotlib and Seaborn are imported.

Graph Creation:
A bar chart is created using sns.barplot with students on x-axis and scores on y-axis.

Styling:
The 'viridis' palette is applied for visualization.

```python
import matplotlib.pyplot as plt
import seaborn as sns

# Simple Bar Chart
plt.figure(figsize=(8, 4))
sns.barplot(x='Student', y='Score', data=df, palette='viridis')
plt.title('Student Performance')
plt.show()
```

```
/tmp/ipython-input-336/4105727942.py:6: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.barplot(x='Student', y='Score', data=df, palette='viridis')

<Figure size 800x400 with 1 Axes>
```

Figure 1.11